



GENERACIÓN DE EVIDENCIAS Y REPORTE DE RESULTADOS

Valeria Hernández Cortés

Grupo IDGS17

Reporte Técnico de Resultados: Monitoreo con Seq

Índice

Reporte Técnico de Resultados: Monitoreo con Seq	1
Índice	1
Objetivo	2
Introducción	3
Configuración del sistema de logging	4
Configuración de Serilog	4
Configuración de Seq	4
Configuración del almacenamiento de logs	4
Configuración del CorrelationId	5
Configuración del Exception Middleware	5
Instrumentación de Program.cs	5
Middleware de CorrelationId	6
Exception Middleware	6
Instrumentación de los controladores	6
Descripción de las pruebas ejecutadas	7
Navegación general	7
Pruebas de búsquedas	7
Pruebas de autenticación	7
Pruebas de comentarios	8
Pruebas de la API	8
Pruebas de excepciones	9
Generación de eventos Warning	10
Resultados obtenidos	11
Estadísticas obtenidas	11
Conclusiones	12

Objetivo

Generar un volumen representativo de registros de eventos utilizando la aplicación VulnerableApp, analizar la información obtenida y elaborar un reporte técnico que documente los resultados alcanzados para evaluar el funcionamiento de la estrategia de logging y la trazabilidad de las solicitudes dentro de la aplicación.

Introducción

Una estrategia de logging solo resulta útil cuando la información registrada permite conocer qué está ocurriendo dentro de una aplicación, detectar errores, reconstruir eventos y analizar posibles incidentes de seguridad. En esta práctica se ejecutaron diferentes escenarios funcionales y de seguridad con el propósito de generar una gran cantidad de registros y validar el funcionamiento de la infraestructura de monitoreo implementada.

Durante las pruebas se realizaron operaciones de navegación, autenticación, búsquedas, consumo de la API, generación de excepciones y simulación de ataques comunes, como intentos de SQL Injection, Cross-Site Scripting (XSS) y fuerza bruta. Posteriormente, toda la información generada fue analizada mediante Seq, lo que permitió comprobar cómo una estrategia de logging bien implementada facilita el monitoreo de la aplicación, la detección de anomalías y el seguimiento de cada solicitud mediante el uso del CorrelationId.

Configuración del sistema de logging

Para esta práctica se reemplazó el sistema de logging predeterminado de ASP.NET Core por una solución basada en Serilog y Seq, con el objetivo de generar registros estructurados, fáciles de consultar y con mayor información de contexto.

Configuración de Serilog

Se configuró Serilog como el proveedor principal de logging utilizando `LoggerConfiguration`. Además, cada evento se enriqueció automáticamente con información adicional, como el contexto de la aplicación y el nombre del equipo, permitiendo obtener registros más completos y facilitar su análisis posterior.

Configuración de Seq

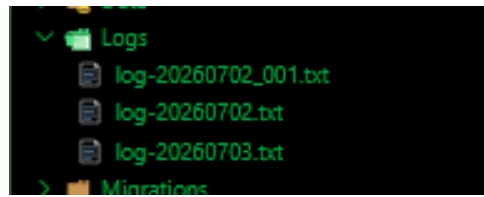
Se integró Seq como plataforma para centralizar todos los registros generados por la aplicación. Gracias a esta integración fue posible visualizar los eventos en tiempo real, filtrarlos por nivel de severidad y realizar búsquedas específicas para identificar errores, advertencias o comportamientos sospechosos.

```
C# Program.cs

1  using Microsoft.EntityFrameworkCore;
2  using VulnerableApp.Data;
3  using Microsoft.EntityFrameworkCore.Diagnostics;
4  using Serilog;      "Serilog": Unknown word.
5  using System.Diagnostics;
6
7  var builder = WebApplication.CreateBuilder(args);
8
9  Log.Logger = new LoggerConfiguration()
10     .ReadFrom.Configuration(builder.Configuration)
11     .WriteTo.Console()
12     .WriteTo.File("Logs/log-.txt", rollingInterval: RollingInterval.Day)
13     .WriteTo.Seq("http://localhost:5341")
14     .Enrich.FromLogContext()
15     .Enrich.WithMachineName() // (enriquecedor) "enriquecedor": Unknown word.
16     .CreateLogger();
17
18  builder.Host.UseSerilog();      "Serilog": Unknown word.
19
```

Configuración del almacenamiento de logs

Se configuró el almacenamiento de los registros en archivos de texto dentro de la carpeta `Logs`, utilizando una rotación automática diaria para mantener organizados los archivos generados.



Configuración del CorrelationId

Se desarrolló un middleware encargado de generar un identificador único (CorrelationId) para cada solicitud HTTP recibida. Este identificador se agregó tanto al contexto de Serilog como a los encabezados de respuesta, permitiendo relacionar todos los eventos pertenecientes a una misma petición y facilitar su seguimiento durante el análisis.

```
51 app.Use(async (context, next) =>
52 {
53     var cid = Guid.NewGuid().ToString();
54     context.Response.Headers["X-Correlation-ID"] = cid;
55
56     // Empuja el CorrelationId al contexto de Serilog para que Seq lo atrape "Empuja": Unknown word.
57     using (Serilog.Context.LogContext.PushProperty("CorrelationId", cid)) "Serilog": Unknown word.
58     {
59         await next(context);
60     }
61 });
62
```

Configuración del Exception Middleware

Se implementó un middleware para manejar de forma centralizada las excepciones no controladas. Su función consiste en capturar cualquier error que ocurra durante la ejecución de la aplicación, registrarlo en los logs y devolver una respuesta HTTP 500 al cliente sin exponer información sensible del sistema.

```
33 app.Use(async (context, next) =>
34 {
35     try
36     {
37         await next(context);
38     }
39     catch (Exception ex)
40     {
41         var logger = context.RequestServices.GetRequiredService<ILogger<Program>>();
42         logger.LogError(ex, "Unhandled Exception: Error no controlado interceptado por Middleware"); "controlado": Unknown word.
43         context.Response.StatusCode = 500;
44         await context.Response.WriteAsync("Ocurrió un error interno en el servidor."); "Ocurrió": Unknown word.
45     }
46 });
47
```

Instrumentación de Program.cs

El archivo Program.cs fue modificado para registrar Serilog como proveedor principal de logging e incorporar los middlewares personalizados dentro del pipeline de la aplicación antes del mapeo de rutas.

Middleware de CorrelationId

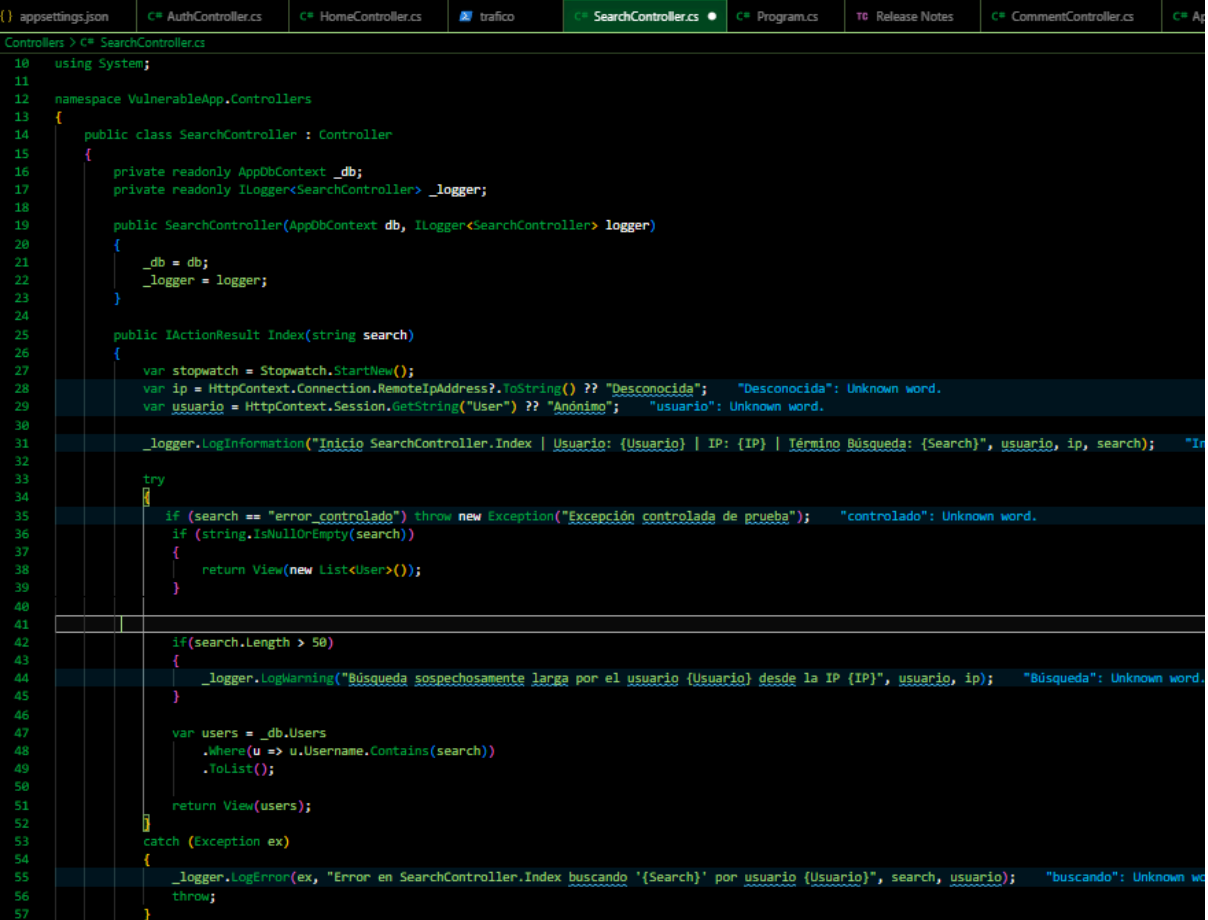
Este middleware permite asignar un identificador único a cada solicitud, haciendo posible rastrear todo su recorrido dentro de la aplicación y localizar posteriormente cada uno de los eventos relacionados.

Exception Middleware

Se implementó para registrar de forma automática cualquier excepción no controlada, evitando que los errores pasaran desapercibidos y facilitando su análisis posterior.

Instrumentación de los controladores

Los principales controladores de la aplicación (AuthController, SearchController, ApiController, HomeController y ApiController) fueron instrumentados agregando registros en las operaciones más importantes. También se utilizó la clase Stopwatch para medir el tiempo de ejecución de cada solicitud y registrar eventos de tipo Information, Warning y Error dependiendo del resultado de cada operación.

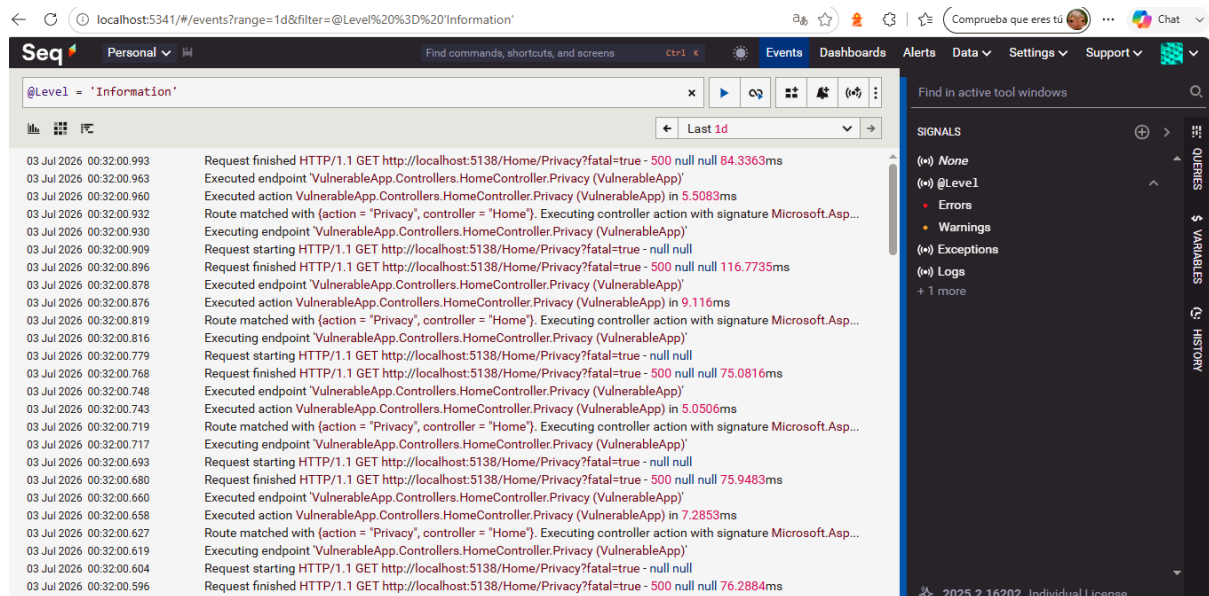


```
10 using System;
11
12 namespace VulnerableApp.Controllers
13 {
14     public class SearchController : Controller
15     {
16         private readonly AppDbContext _db;
17         private readonly ILogger<SearchController> _logger;
18
19         public SearchController(AppDbContext db, ILogger<SearchController> logger)
20         {
21             _db = db;
22             _logger = logger;
23         }
24
25         public IActionResult Index(string search)
26         {
27             var stopwatch = Stopwatch.StartNew();
28             var ip = HttpContext.Connection.RemoteIpAddress?.ToString() ?? "Desconocida"; "Desconocida": Unknown word.
29             var usuario = HttpContext.Session.GetString("User") ?? "Anónimo"; "usuario": Unknown word.
30
31             _logger.LogInformation("Inicio SearchController.Index | Usuario: {Usuario} | IP: {IP} | Término Búsqueda: {Search}", usuario, ip, search); "In
32
33             try
34             {
35                 if (search == "error_controlado") throw new Exception("Excepción controlada de prueba"); "controlado": Unknown word.
36                 if (string.IsNullOrEmpty(search))
37                 {
38                     return View(new List<User>());
39                 }
40
41
42                 if (search.Length > 50)
43                 {
44                     _logger.LogWarning("Búsqueda sospechosamente larga por el usuario {Usuario} desde la IP {IP}", usuario, ip); "Búsqueda": Unknown word.
45                 }
46
47                 var users = _db.Users
48                     .Where(u => u.Username.Contains(search))
49                     .ToList();
50
51                 return View(users);
52             }
53             catch (Exception ex)
54             {
55                 _logger.LogError(ex, "Error en SearchController.Index buscando '{Search}' por usuario {Usuario}", search, usuario); "buscando": Unknown we
56                 throw;
57             }
58         }
59     }
60 }
```

Descripción de las pruebas ejecutadas

Navegación general

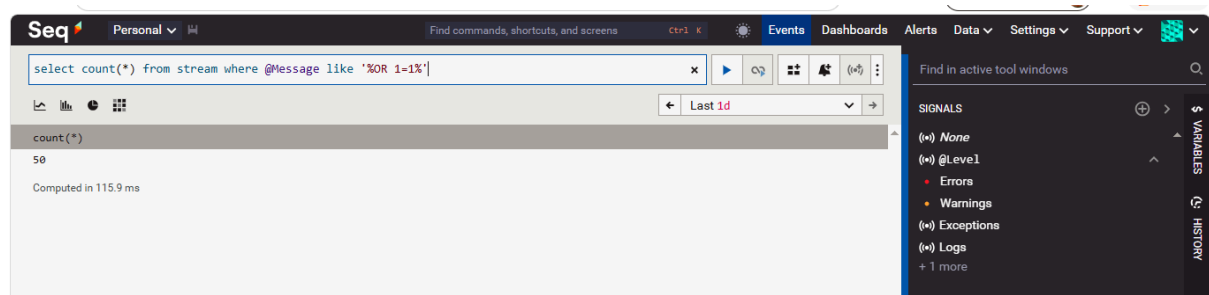
Se realizaron múltiples accesos consecutivos a la página principal y al resto de los controladores disponibles con el objetivo de generar tráfico constante y verificar que cada solicitud registrara correctamente su CorrelationId.



Pruebas de búsquedas

Para evaluar el comportamiento del módulo de búsqueda se realizaron los siguientes escenarios:

- 100 búsquedas con parámetros válidos.
- 20 búsquedas utilizando parámetros vacíos.
- 20 búsquedas con caracteres especiales.
- 20 búsquedas simulando intentos de SQL Injection mediante cadenas como ' OR 1=1 --.



Pruebas de autenticación

Se ejecutaron distintos escenarios relacionados con el proceso de inicio de sesión:

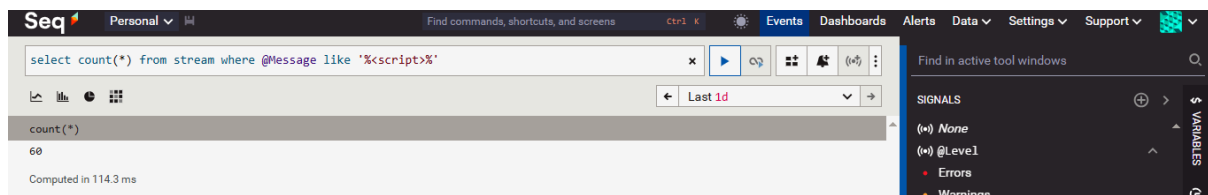
- 50 autenticaciones exitosas.
- 100 intentos fallidos.
- Intentos utilizando usuarios inexistentes.
- Verificación de que las contraseñas nunca fueran almacenadas dentro de los registros.

03 Jul 2026 00:30:36.070 Inicio AuthController.Login (POST) | Intento de usuario: **hacker** | IP: ::1 | Password: [REDACTED]

Event	Level (Information)	Type (0x95DD4218)	Trace (360e...)	Export
✓ x	ActionId	87f7d780-fe5d-4252-a203-a19b3a8458a4		
✓ x	ActionName	VulnerableApp.Controllers.AuthController.Login (VulnerableApp)		
✓ x	ConnectionId	0HNMOGPVHHNSL		
✓ x	CorrelationId	2f4d31b0-d6d1-461c-8edf-3af4623f3bbb		
✓ x	IP	::1		
✓ x	MachineName	DESKTOP-H4CTBTI		
✓ x	RequestId	0HNMOGPVHHNSL:0000012B		
✓ x	RequestPath	/Auth/Login		
✓ x	SourceContext	VulnerableApp.Controllers.AuthController		
✓ x	Username	hacker		

Pruebas de comentarios

Se realizaron pruebas sobre el módulo de comentarios agregando entradas válidas y simulando posibles ataques XSS mediante etiquetas `<script>`, verificando que el sistema registrara las advertencias correspondientes.



Pruebas de la API

Se consumieron de forma repetitiva los diferentes endpoints disponibles, además de realizar consultas a recursos inexistentes e identificadores inválidos para comprobar el comportamiento de la aplicación y la generación de registros asociados.

Seq	
Personal	
Find commands, shortcuts, and screens	
Events	
Dashboards	
Alerts	
Data	
Settings	
Support	
select count(*) as Total from stream where Path is not null group by Path order by Total desc	
Last 1d	
Path	Total
/api/users	2818
/Search	2360
/Auth/Login	1815
/Home/Privacy	1686
	1626
/Comment	1061
/Comment/AddComment	662
/	234
/css/site.css	195
/VulnerableApp.styles.css	195
/lib/bootstrap/dist/css/bootstrap.min.css	195
/lib/jquery/dist/jquery.min.js	188
/js/site.js	188
/lib/bootstrap/dist/js/bootstrap.bundle.min.js	188
/api/recurso_falso	140
/auth/login	91
/search	66
/Auth/Dashboard	56
/favicon.ico	44
/.well-known/appspecific/com.chrome.devtools.json	28

Pruebas de excepciones

Se provocaron tanto excepciones controladas como no controladas con el objetivo de validar el funcionamiento del Exception Middleware y comprobar que todos los errores fueran registrados correctamente.

03 Jul 2026 00:32:00.750

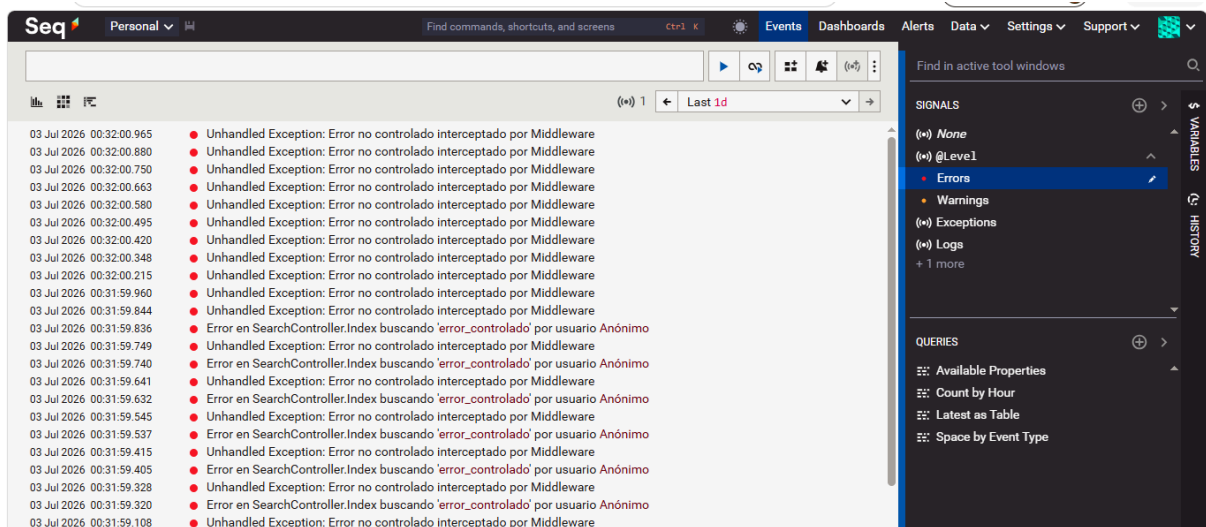
Unhandled Exception: Error no controlado interceptado por Middleware

Event Level (Error) Type (0x665C5DCC) Trace (6d9b...) Export

✓ x ConnectionId 0HNHOPGVHHNSL
 ✓ x MachineName DESKTOP-H4CTBTI
 ✓ x RequestId 0HNHOPGVHHNSL:00000350
 ✓ x RequestPath /Home/Privacy
 ✓ x SourceContext Program

```

System.Exception: Error catastrófico no controlado
   at VulnerableApp.Controllers.HomeController.Privacy() in
C:\Users\valec\OneDrive\Documentos\git\VulnerableApp\Controllers\HomeController.cs:line 45
   at lambda_method103(Closure, Object, Object[])
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeActionMethodAsync()
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeNextActionFilterAsync()
--- End of stack trace from previous location ---
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.Rethrow(ActionExecutedContextSealed
context)
   at Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker.InvokeInnerFilterAsync()
--- End of stack trace from previous location ---
   at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.
<InvokeNextResourceFilter>g__Awaited|25_0(ResourceInvoker invoker, Task lastTask, State next, Scope scope,
Object state, Boolean isCompleted)
   at Microsoft.AspNetCore.Mvc.Infrastructure.ResourceInvoker.Rethrow(ResourceExecutedContextSealed context)
  
```



Generación de eventos Warning

Se diseñaron diferentes escenarios para generar eventos de tipo Warning, incluyendo intentos de autenticación inválidos, accesos no autorizados y entradas con posibles cadenas maliciosas.

03 Jul 2026 00:31:43.803	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		
03 Jul 2026 00:31:43.719	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		
03 Jul 2026 00:31:43.640	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		
03 Jul 2026 00:31:43.543	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		
Event Level (Warning) Type (0x169D543D) Trace (1e6e...) Export <table> <tr> <td>ActionId</td><td>a5ca4605-b576-423c-9824-d100bcba3ac</td></tr> <tr> <td>ActionName</td><td>VulnerableApp.Controllers.ApiController.GetAllUsers (VulnerableApp)</td></tr> <tr> <td>ConnectionId</td><td>0HNMPGVHHNSL</td></tr> <tr> <td>CorrelationId</td><td>a082b89f-95ef-4e63-8fc3-b0340e8bb6f4</td></tr> <tr> <td>IP</td><td>::1</td></tr> <tr> <td>MachineName</td><td>DESKTOP-H4CTBTI</td></tr> <tr> <td>RequestId</td><td>0HNMPGVHHNSL:00000290</td></tr> <tr> <td>RequestPath</td><td>/api/users</td></tr> <tr> <td>SourceContext</td><td>VulnerableApp.Controllers.ApiController</td></tr> </table>		ActionId	a5ca4605-b576-423c-9824-d100bcba3ac	ActionName	VulnerableApp.Controllers.ApiController.GetAllUsers (VulnerableApp)	ConnectionId	0HNMPGVHHNSL	CorrelationId	a082b89f-95ef-4e63-8fc3-b0340e8bb6f4	IP	::1	MachineName	DESKTOP-H4CTBTI	RequestId	0HNMPGVHHNSL:00000290	RequestPath	/api/users	SourceContext	VulnerableApp.Controllers.ApiController
ActionId	a5ca4605-b576-423c-9824-d100bcba3ac																		
ActionName	VulnerableApp.Controllers.ApiController.GetAllUsers (VulnerableApp)																		
ConnectionId	0HNMPGVHHNSL																		
CorrelationId	a082b89f-95ef-4e63-8fc3-b0340e8bb6f4																		
IP	::1																		
MachineName	DESKTOP-H4CTBTI																		
RequestId	0HNMPGVHHNSL:00000290																		
RequestPath	/api/users																		
SourceContext	VulnerableApp.Controllers.ApiController																		
03 Jul 2026 00:31:43.456	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		
03 Jul 2026 00:31:43.350	Intento de acceso no autorizado a la API GetAllUsers desde IP: ::1																		

Resultados obtenidos

Después de ejecutar todas las pruebas, la infraestructura de logging respondió correctamente al volumen de solicitudes generado. Tanto los eventos normales de la aplicación como aquellos relacionados con errores y posibles incidentes de seguridad fueron registrados de manera estructurada, permitiendo consultar toda la información desde Seq sin pérdida de datos. Esto facilitó el análisis del comportamiento de la aplicación y la identificación de los eventos más relevantes durante las pruebas.

Estadísticas obtenidas

1. Eventos Information registrados: 16,600.
2. Eventos Warning registrados: 720.
3. Eventos Error registrados: 63.
4. Controlador con mayor número de registros: a nivel de infraestructura, `Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker` con 12,337 registros; a nivel de aplicación, `VulnerableApp.Controllers.ApiController` con 1,208 registros.
5. Endpoint con mayor número de solicitudes: `/api/users`, con 2,816 solicitudes.
6. Dirección IP con mayor actividad: `::1`, con 2,192 registros.
7. Intentos de autenticación fallidos: 304.
8. Intentos de SQL Injection detectados: 50.
9. Posibles intentos de XSS registrados: 50.
10. Solicitud con mayor tiempo de ejecución: 12,158 ms.
11. Búsqueda mediante `CorrelationId`: fue posible localizar toda la información correspondiente a la solicitud con identificador `0de169ad-a306-4d2b-848f-509c9106c964`, reconstruyendo completamente su ejecución.

The screenshot displays the Seq application interface. The top navigation bar includes 'Personal', 'Find commands, shortcuts, and screens', 'Events', 'Dashboards', 'Alerts', 'Data', 'Settings', and 'Support'. The main content area shows a log entry for a request with `CorrelationId = '0de169ad-a306-4d2b-848f-509c9106c964'`. The log entry details the execution of the endpoint `VulnerableApp.Controllers.HomeController.Privacy (VulnerableApp)` and the execution of the controller factory for `VulnerableApp.Controllers.HomeController (VulnerableApp)`. The log entry also shows the execution plan of exception filters, action filters, and resource filters. The log entry is timestamped '03 Jul 2026 00:32:00.963'.

On the right side of the interface, there is a sidebar with 'SIGNALS' and 'QUERIES' sections. The 'SIGNALS' section shows a list of signals: 'None', '@Level', 'Errors', 'Warnings', 'Exceptions', and 'Logs'. The 'QUERIES' section shows a list of queries: 'Available Properties', 'Count by Hour', 'Latest as Table', and 'Space by Event Type'.

Conclusiones

La implementación de Serilog y Seq permitió contar con un sistema de monitoreo mucho más completo que el logging tradicional. Durante las pruebas fue posible registrar correctamente todas las solicitudes realizadas, identificar errores, detectar intentos de ataques como SQL Injection, XSS y fuerza bruta, además de verificar que la información sensible, como las contraseñas, nunca fuera almacenada en los registros.

Por otra parte, el uso del CorrelationId facilitó el seguimiento de cada solicitud desde que ingresó a la aplicación hasta que finalizó su procesamiento, haciendo mucho más sencillo localizar eventos específicos dentro de miles de registros. En conjunto, los resultados obtenidos demuestran que la estrategia de logging implementada proporciona una mejor visibilidad sobre el funcionamiento de la aplicación y representa una herramienta importante tanto para el monitoreo como para el análisis y la detección de incidentes de seguridad.